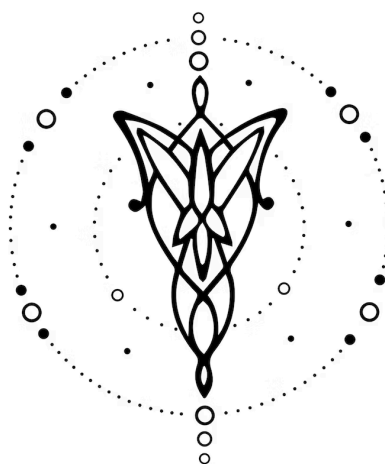




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de marzo de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Facundo	Ponce	65288	faponce@itba.edu.ar
Matias	Holgado	64727	mholgado@itba.edu.ar
Matias	Sanchez Novelli	65741	msancheznovelli@itba.edu.ar
Lucas	Rodriguez	63335	lrodriguenahuel@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/fa-ponce/tp-tla-2026>

3. Dominio

Desarrollar un lenguaje que permita describir, versionar y reutilizar transformaciones sobre archivos de formato CSV (Comma separated values): carga, filtrado fila por fila, derivación de nuevas columnas, proyección de un subconjunto de columnas, ordenamiento, truncado y guardado del resultado.

Hoy estas tareas de limpieza y extracción de datos se resuelven con scripts en lenguajes de propósito general (pandas, R) o con cadenas de herramientas de línea de comandos (`awk`, `cut`, `sort`, `sed`). Ese enfoque no escala: cada analista reescribe la misma lógica, los scripts quedan sin tests, la transformación se mezcla con detalles de parsing y de I/O, y no hay una forma uniforme de compartir una pipeline entre proyectos.

Surge entonces una necesidad arquitectural: contar con una especificación declarativa de la transformación, independiente de la mecánica de lectura/escritura del CSV, del manejo de tipos por columna y del orden concreto de las operaciones. TabularDSL es esa especificación: el programador declara qué transformación quiere y el compilador se encarga de cómo ejecutarla. Al ser el lenguaje una interfaz estable, el compilador puede optimizar el motor de filas/columnas subyacente sin afectar los programas ya escritos, y esos programas pueden versionarse, testearse y reutilizarse con las herramientas habituales del desarrollo de software.

Para acotar la complejidad del proyecto:

- Toda la comunicación y el procesamiento ocurren en la memoria del propio programa. No hay acceso a bases de datos, red ni servicios externos.
- El artefacto de entrada del compilador es un programa TabularDSL (texto). El o los archivos CSV referenciados por `LOAD` son datos que el compilador lee al ejecutar el programa.
- El artefacto de salida es un único archivo CSV con el resultado de la transformación.
- El esquema (nombres y tipos de columna) se infiere del archivo de entrada; el programador lo declara.
- Los valores de celda se modelan con tres tipos: `'number'`, `'string'` y `'boolean'`.

La implementación satisfactoria de este lenguaje permitiría a un equipo de datos mantener un catálogo de transformaciones probadas y reutilizables, reduciendo el trabajo repetitivo y los errores de la limpieza manual de datos.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

1. Un programa comienza obligatoriamente con una instrucción `LOAD "<ruta>"`, cuyo argumento es un literal de tipo string, y termina obligatoriamente con `SAVE "<ruta>"`.
2. El esquema del CSV de entrada se infiere automáticamente. Las columnas se referencian por el nombre del encabezado dentro de cualquier expresión.
3. Los valores de celda podrán ser de tipo number, string o boolean. El tipo de cada columna se infiere del archivo.

4. Filtrado de registros (FILTER): evaluación de una expresión booleana fila por fila, con operadores relacionales (=, !=, <, <=, >, >=), operadores lógicos (AND, OR, NOT) y paréntesis para forzar precedencia.
5. Transformación (MUTATE): creación de una nueva columna a partir de una expresión aritmética (+, -, *, /) sobre columnas existentes, literales numéricos, constantes y llamadas a funciones, con la precedencia habitual y paréntesis.
6. Transformación condicional: la expresión de un MUTATE puede ser IF <condición> THEN <expr> ELSE <expr>. Ambas ramas son obligatorias.
7. Proyección (SELECT): selección de un subconjunto de columnas a retener en la salida.
8. Ordenamiento (SORT BY <columna> ASC[DESC]) y truncado (LIMIT <n>) del resultado.
9. Constantes de script (let <ident> = <expr>;): ligam un identificador a una expresión constante reutilizable en cualquier expresión o condición del programa.
10. Funciones (function <ident>(<params>) { return <expr>; }): fórmulas propias de una sola expresión de retorno, reutilizables dentro de MUTATE y FILTER.
11. Pipelines reutilizables (pipeline <ident>(<params>) { <stages> <requires> }): secuencias parametrizadas de stages (FILTER, MUTATE, SELECT, SORT, LIMIT) reutilizables entre programas.
12. Invocación de pipelines (APPLY <ident>(<args>;)): inserta las stages de un pipeline en el punto del programa donde aparece. La aridad de los argumentos debe coincidir con la de los parámetros del pipeline.
13. Restricciones estáticas (require <condición booleana>;): dentro de un pipeline, acotan el valor de sus parámetros o de las constantes let. Se validan en tiempo de compilación.
14. Iteración sobre variantes (for <ident> in [<v1>, <v2>, ...] { <programa> }): ejecuta el mismo programa una vez por cada valor de la lista, ligando el identificador a cada variante. Permite correr la misma transformación con distintos parámetros sin repetir el programa.
15. Comentarios de línea con “//”, ignorados por el compilador.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

1. Carga y guardado sin modificaciones.
2. Proyección simple de una columna.
3. Proyección de múltiples columnas.
4. Filtrado con operador relacional simple.
5. Filtrado con operadores lógicos y paréntesis para precedencia.
6. Nueva columna con operación aritmética.
7. Nueva columna derivada con IF/THEN/ELSE
8. Ordenamiento ascendente en un programa y descendente en otro.
9. Pipeline lineal completo sin errores.
10. function con un parámetro, usada dentro de un MUTATE.

Además, los siguientes casos de prueba de **rechazo**:

1. Falta la instrucción LOAD obligatoria inicial.
2. Falta la instrucción SAVE obligatoria final.
3. El argumento de LOAD no es un string.
4. Expresión booleana mal formada (falta el operando derecho).
5. SELECT sin identificadores de columna.
6. Palabra reservada inexistente en la gramática.
7. Operación entre tipos incompatibles (aritmética sobre string).
8. MUTATE condicional sin rama ELSE (viola la exhaustividad).
9. Referencia a una columna inexistente en el esquema inferido.
10. APPLY con una aridad distinta a la del pipeline.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver un flujo pequeño en el cual se carga un archivo, filtra los usuarios mayores de edad que están activos y guarda únicamente la columna de correos electrónicos.

```
// Cargar el archivo de usuarios:
LOAD "users.csv"

// Quedarse solo con los adultos activos:
FILTER age >= 18 AND status = "active"

// Proyectar una sola columna y guardar:
SELECT email
SAVE "active_adults.csv"
```

Código 6.1

En el **Cód. (6.2)**, se define una transformación reutilizable que aplica un impuesto y proyecta columnas, la invoca sobre el catálogo de productos, ordena de mayor a menor y guarda el top 10. La lógica de impuesto queda extraída en una abstracción reutilizable en vez de estar escrita inline.

```
pipeline aplicar_impuesto(tasa) {
  MUTATE precio_final = precio_base * (1 + tasa)
  SELECT nombre, precio_final
  require tasa >= 0 AND tasa <= 1;
}

LOAD "products.csv"
APPLY aplicar_impuesto(0.21)
SORT BY precio_final DESC
```

```
LIMIT 10
SAVE "top_products.csv"
```

Código 6.2:

En el **Cód. (6.3)**, se define una constante reutilizable (`tasa_base`), una función (`recargo`) usada dentro de la rama `THEN` de un `MUTATE` condicional que suma un recargo a los productos caros, un `require` que acota el parámetro `tasa` del pipeline entre 0 y 1, y un `for` que corre la misma transformación con tres tasas de impuesto distintas sin repetir el programa.

```
let tasa_base = 0.21;

function recargo(precio) {
  return precio * 0.05;
}

pipeline aplicar_impuesto(tasa) {
  MUTATE precio_final = IF precio > 1000
                        THEN precio * (1 + tasa) + recargo(precio)
                        ELSE precio * (1 + tasa)
  SELECT nombre, precio_final
  require tasa >= 0 AND tasa <= 1;
}

for tasa in [0.10, tasa_base, 0.27] {
  LOAD "products.csv"
  APPLY aplicar_impuesto(tasa)
  SORT BY precio_final DESC
  LIMIT 10
  SAVE "top_products.csv"
}
```

Código 6.3: